

T35-项目文档

成员：韩艺成，孙鲁，周屿杨，顾益铭

选题：面向XXX场景的深度学习模型测试技术

场景

某图像分类工具正在开发，旨在通过自动化系统识别和分类用户上传的图片。该工具使用**ResNet18**模型对图片进行分类，并根据**CIFAR-10**数据集的10个类别提供相应的图片标签。然而，图像识别应用可能会遇到多种挑战，包括拍摄角度变化、图像模糊、噪声等扰动，这些因素可能影响识别模型的性能。因此，确保该系统在不同条件下具有足够的鲁棒性，准确性等多种指标，且能够精准、公正地分类各种图片，对其实际部署非常重要。

主要项目结构

```
1 | .
2 | └─ data/
3 |   └─ (存放数据集和本地模型)
4 |
5 | └─ src/
6 |   └─ data.py           # 加载 CIFAR-10 数据集，划分数据集
7 |   └─ model.py          # 包括加载模型、训练模型、保存模型的方法
8 |   └─ test_utils.py      # 包括各个测试方法的评估函数
9 |   └─ transforms.py      # 包括数据图像不同处理的函数
10 |
11 | └─ tests/
12 |   └─ conftest.py        # `pytest` 框架编写的测试配置文件
13 |   └─ test_accuracy.py  # 准确性测试函数
14 |   └─ test_fairness.py  # 公平性测试函数
15 |   └─ test_metamorphic.py # 蜕变测试函数
16 |   └─ test_performance.py # 性能测试函数
17 |   └─ test_robustness.py # 鲁棒性测试函数
18 |   └─ train_model.py    # 训练模型函数
19 |
```

项目运行过程

准备工作

准备数据集：请把所给的data文件夹放在项目的根目录下

运行测试

- 准确性测试： `pytest -s ./tests/test_accuracy.py`
- 公平性测试： `pytest -s ./tests/test_fairness.py`
- 蜕变测试： `pytest -s ./tests/metamorphic.py`
- 性能测试： `pytest -s ./tests/performance.py`
- 鲁棒性测试： `pytest -s ./tests/robustness.py`

运行过程

1. 使用data.py中的函数加载 CIFAR-10 数据集，并创建训练集和测试集的数据加载器
2. 如果存在预训练的模型文件（resnet_model.pth），则加载该模型；否则，训练一个新的 ResNet 模型并将训练好的模型保存到本地。（模型训练使用 GPU（如果可用），否则使用 CPU）
3. 运行测试函数
4. 结果保存到chart文件夹中

测试方法介绍

准确性测试

参数

- **无参数**：此函数不接受任何参数。

返回值

- **无返回值**：此函数不返回任何值。它通过 `pytest` 的断言机制来验证模型的准确性是否符合预期。

测试过程

1. 加载数据：
 - 使用 `load_cifar10` 函数加载 CIFAR-10 数据集，并创建训练集和测试集的数据加载器（`train_loader` 和 `test_loader`）。
2. 加载并训练模型：
 - 如果存在预训练的模型文件（`resnet_model.pth`），则加载该模型；否则，训练一个新的 ResNet 模型并将训练好的模型保存到本地。
 - 模型训练使用 GPU（如果可用），否则使用 CPU。
3. 评估原始数据的准确性：
 - 使用 `evaluate_accuracy` 函数评估模型在原始测试数据上的分类准确性，并打印结果。
4. 添加噪声并评估：
 - 对测试数据添加随机噪声（噪声水平为 0.1），并创建新的数据加载器 `noisy_loader`。
 - 评估模型在噪声数据上的分类准确性，并打印结果。
5. 添加模糊并评估：
 - 对测试数据应用高斯模糊（核大小为 5），并创建新的数据加载器 `blurred_loader`。
 - 评估模型在模糊数据上的分类准确性，并打印结果。
6. 翻转图像并评估：
 - 对测试数据进行水平翻转，并创建新的数据加载器 `flipped_loader`。
 - 评估模型在翻转图像上的分类准确性，并打印结果。
7. 断言准确性：

- 通过 `assert` 语句确保模型在原始数据、噪声数据、模糊数据和翻转图像上的准确性都高于设定的最低阈值（原始准确性 > 70%，其他情况下 > 50%）。如果任何一个准确性低于阈值，测试将失败并抛出异常。

源代码

```
1 def test_model_accuracy_with_variations():
2     """
3     Tests the accuracy of the model with variations like noise, blur, and
4     flips.
5     性能测试：检测模型在不同数据变异下的正确率
6     """
7     # 加载数据
8     train_loader, test_loader = load_cifar10(batch_size=32)
9
10    # 加载并训练模型
11    model = load_resnet()
12    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
13    # model = train_model(model, train_loader, epochs=10, lr=0.001,
14    device=device)
15
16    model_path = "./data/resnet_model.pth"
17    if os.path.exists(model_path):
18        print(f>Loading model from {model_path}...")
19        model = load_model(model, path=model_path, device=device)
20
21    else:
22        print("Training model...")
23        model = train_model(model, train_loader, epochs=3, lr=0.001,
24        device=device)
25        save_model(model, path=model_path) # 保存模型
26        print(f"Model saved to {model_path}.")
27
28    # 测试原始数据的准确性
29    original_accuracy = evaluate_accuracy(model, test_loader, device)
30    print(f"Original Test Accuracy: {original_accuracy * 100:.2f}%")
31
32    # 对测试数据添加噪声并评估
33    test_images, test_labels = next(iter(test_loader))
34    noisy_images = add_noise(test_images.numpy(), noise_level=0.1)
35    noisy_dataset = torch.utils.data.TensorDataset(
36        torch.tensor(noisy_images), torch.tensor(test_labels)
37    )
38    noisy_loader = torch.utils.data.DataLoader(noisy_dataset, batch_size=32)
39    noisy_accuracy = evaluate_accuracy(model, noisy_loader, device)
40    print(f"Test Accuracy with Noise: {noisy_accuracy * 100:.2f}%")
41
42    # 对测试数据添加模糊并评估
43    blurred_images = add_blur(test_images.numpy(), kernel_size=5)
44    blurred_dataset = torch.utils.data.TensorDataset(
45        torch.tensor(blurred_images), torch.tensor(test_labels)
46    )
```

```

46     blurred_loader = torch.utils.data.DataLoader(blurred_dataset,
batch_size=32)
47     blurred_accuracy = evaluate_accuracy(model, blurred_loader, device)
48     print(f"Test Accuracy with Blur: {blurred_accuracy * 100:.2f}%")
49
50     # 对测试数据翻转并评估
51     flipped_images = add_flip(test_images.numpy())
52     flipped_dataset = torch.utils.data.TensorDataset(
53         torch.tensor(flipped_images), torch.tensor(test_labels)
54     )
55     flipped_loader = torch.utils.data.DataLoader(flipped_dataset,
batch_size=32)
56     flipped_accuracy = evaluate_accuracy(model, flipped_loader, device)
57     print(f"Test Accuracy with Flips: {flipped_accuracy * 100:.2f}%")
58
59     # 确保准确性达到最低阈值
60     assert original_accuracy > 0.7, "Original accuracy is below the expected
threshold!"
61     assert noisy_accuracy > 0.5, "Accuracy with noise is below the expected
threshold!"
62     assert blurred_accuracy > 0.5, "Accuracy with blur is below the expected
threshold!"
63     assert flipped_accuracy > 0.5, "Accuracy with flips is below the expected
threshold!"

```

公平性测试

参数

- **无参数**：此函数不接受任何参数。

返回值

- **无返回值**：此函数不返回任何值。它通过 `pytest` 的断言机制来验证模型的公平性是否符合预期。

测试过程

1. 统计类别分布：

- 使用 `load_cifar10_fairness` 函数加载测试集，并统计每个类别的样本数量。
- 打印测试集中各个类别的分布情况，帮助开发者了解数据的平衡性。

2. 加载数据：

- 使用 `load_cifar10` 函数加载完整的 CIFAR-10 数据集，创建训练集和测试集的数据加载器（`train_loader` 和 `test_loader`）。

3. 加载并训练模型：

- 如果存在预训练的模型文件（`resnet_model.pth`），则加载该模型；否则，训练一个新的 ResNet 模型并将训练好的模型保存到本地。
- 模型训练使用 GPU（如果可用），否则使用 CPU。

4. 模拟敏感特征标签：

- 定义敏感特征类别索引（例如，类别 0 和类别 1）。这些类别将用于评估模型的公平性。

5. 测试公平性:

- 使用 `evaluate_fairness` 函数评估模型在测试集上的公平性。该函数会生成针对每个敏感类别的公平性报告，包括精度、召回率、F1 分数等指标。

6. 输出公平性报告:

- 对于每个敏感类别，打印详细的公平性报告，展示模型在该类别上的表现。

7. 断言公平性:

- 通过 `assert` 语句确保模型对每个类别的 F1 分数都高于 0.5。如果任何一个类别的 F1 分数低于阈值，测试将失败并抛出异常。

源代码

```
1 def test_model_fairness():
2     # 统计类别分布
3     _, test_loader = load_cifar10_fairness(batch_size=32)
4     all_labels = []
5     for _, labels in test_loader:
6         all_labels.extend(labels.numpy())
7
8     label_counts = Counter(all_labels)
9     print("Label distribution in test set:", label_counts)
10
11    # 加载数据
12    train_loader, test_loader = load_cifar10(batch_size=32)
13
14    # 加载并训练模型
15    model = load_resnet()
16    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
17
18    model_path = "./data/resnet_model.pth"
19    if os.path.exists(model_path):
20        print(f"Loading model from {model_path}...")
21        model = load_model(model, path=model_path, device=device)
22
23    else:
24        print("Training model...")
25        model = train_model(model, train_loader, epochs=3, lr=0.001,
26                             device=device)
27        save_model(model, path=model_path) # 保存模型
28        print(f"Model saved to {model_path}.")
29
30    # 模拟敏感特征标签（例如，类别 0 和类别 1 作为敏感特征）
31    sensitive_feature_indices = [0, 1]
32
33    # 测试公平性
34    fairness_reports = evaluate_fairness(model, test_loader, device,
35                                         sensitive_feature_indices)
36
37    # 输出公平性报告
38    for category, report in fairness_reports.items():
39        print(f"Fairness report for sensitive category {category}:")
40        print(report)
41
42    # 确保模型对每个类别的 F1 分数都高于 0.5
```

```
41     for category, report in fairness_reports.items():
42         assert report["macro avg"]["f1-score"] > 0.5, f"Fairness issue
detected for category {category}!"
```

蜕变测试

参数

- **无参数**：此函数不接受任何参数。

返回值

- **无返回值**：此函数不返回任何值。它通过 `pytest` 的断言机制来验证模型的蜕变属性是否符合预期。

测试过程

1. 加载数据：

- 使用 `load_cifar10` 函数加载 CIFAR-10 数据集，并创建训练集和测试集的数据加载器（`train_loader` 和 `test_loader`）。

2. 加载并训练模型：

- 如果存在预训练的模型文件（`resnet_model.pth`），则加载该模型；否则，训练一个新的 ResNet 模型并将训练好的模型保存到本地。
- 模型训练使用 GPU（如果可用），否则使用 CPU。
- 将模型移动到指定的设备（GPU 或 CPU）上。

3. 获取原始测试数据：

- 从 `test_loader` 中获取一批测试数据（图像和标签），并将它们移动到设备上（GPU 或 CPU）。

4. 创建变换后的数据集（噪声变异）：

- 对原始测试图像添加随机噪声（噪声水平为 0.1），生成新的噪声图像。
- 创建包含噪声图像和原始标签的新数据集 `noisy_dataset`，并将其转换为数据加载器 `noisy_loader`。
- 注意：噪声变换在 CPU 上进行，以避免在 GPU 上处理时的潜在问题。

5. 测试蜕变属性（变换前后输出一致性）：

- 使用 `metamorphic_test` 函数比较模型在原始数据和噪声数据上的输出，计算输出的一致性比例。
- 打印一致性比例，展示模型在噪声变异下的表现。

6. 断言一致性：

- 通过 `assert` 语句确保模型的输出一致性高于 70%。如果一致性低于阈值，测试将失败并抛出异常。

源代码

```
1 def test_model_metamorphic():
2     # 加载数据
3     train_loader, test_loader = load_cifar10(batch_size=32)
4
```

```

5     # 加载并训练模型
6     model = load_resnet()
7     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
8     model.to(device) # 将模型移动到设备上
9     # model = train_model(model, train_loader, epochs=5, lr=0.001,
device=device)
10    model_path = "./data/resnet_model.pth"
11    if os.path.exists(model_path):
12        print(f"Loading model from {model_path}...")
13        model = load_model(model, path=model_path, device=device)
14
15    else:
16        print("Training model...")
17        model = train_model(model, train_loader, epochs=3, lr=0.001,
device=device)
18        save_model(model, path=model_path) # 保存模型
19        print(f"Model saved to {model_path}.")
20
21
22    # 获取原始测试数据
23    test_images, test_labels = next(iter(test_loader))
24    test_images, test_labels = test_images.to(device), test_labels.to(device)
# 将输入数据移动到设备上
25
26    # 创建变换后的数据集（噪声变异）
27    noisy_images = add_noise(test_images.cpu().numpy(), noise_level=0.1,
dtype=np.float32) # 在CPU上进行噪声变换
28    noisy_dataset = torch.utils.data.TensorDataset(
29        torch.tensor(noisy_images).to(device),
torch.tensor(test_labels).to(device)
30    )
31    noisy_loader = torch.utils.data.DataLoader(noisy_dataset, batch_size=32)
32
33    # 测试蜕变属性（变换前后输出一致性）
34    consistency = metamorphic_test(model, test_loader, noisy_loader, device)
35    print(f"Consistency between original and noisy data: {consistency *
100:.2f}%")
36
37    # 确保一致性高于 90%
38    assert consistency > 0.7, "Metamorphic testing failed: consistency too
low!"

```

性能测试

参数

- **无参数**：此函数不接受任何参数。

返回值

- **无返回值**：此函数不返回任何值。它通过 `pytest` 的断言机制来验证模型的推理性能是否符合预期。

测试过程

1. 加载数据:

- 使用 `load_cifar10` 函数加载 CIFAR-10 数据集，并创建训练集和测试集的数据加载器 (`train_loader` 和 `test_loader`)。

2. 加载并训练模型:

- 如果存在预训练的模型文件 (`resnet_model.pth`)，则加载该模型；否则，训练一个新的 ResNet 模型并将训练好的模型保存到本地。
- 模型训练使用 GPU (如果可用)，否则使用 CPU。
- 将模型移动到指定的设备 (GPU 或 CPU) 上。

3. 测试性能:

- 使用 `evaluate_performance` 函数评估模型在原始测试数据上的平均推理时间，并打印结果。

4. 添加噪声并评估:

- 对测试数据添加随机噪声 (噪声水平为 0.1)，生成新的噪声图像。
- 创建包含噪声图像和原始标签的新数据集 `noisy_dataset`，并将其转换为数据加载器 `noisy_loader`。
- 评估模型在噪声数据上的平均推理时间，并打印结果。

5. 添加模糊并评估:

- 对测试数据应用高斯模糊 (核大小为 5)，生成新的模糊图像。
- 创建包含模糊图像和原始标签的新数据集 `blurred_dataset`，并将其转换为数据加载器 `blurred_loader`。
- 评估模型在模糊数据上的平均推理时间，并打印结果。

6. 翻转图像并评估:

- 对测试数据进行旋转 (假设 `add_rotation` 函数实现的是图像的旋转操作)，生成新的旋转图像。
- 创建包含旋转图像和原始标签的新数据集 `flipped_dataset`，并将其转换为数据加载器 `flipped_loader`。
- 评估模型在旋转图像上的平均推理时间，并打印结果。

7. 断言性能:

- 通过 `assert` 语句确保模型在原始数据、噪声数据、模糊数据和旋转图像上的推理时间都在设定的合理范围内。如果任何一个推理时间超过阈值，测试将失败并抛出异常。

源代码

```
1 def test_model_performance():
2     # 加载数据
3     train_loader, test_loader = load_cifar10(batch_size=32)
4
5     # 加载并训练模型
6     model = load_resnet()
7     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```



```

8     # model = train_model(model, train_loader, epochs=5, lr=0.001,
device=device)
9     model_path = "./data/resnet_model.pth"
10    if os.path.exists(model_path):
11        print(f"Loading model from {model_path}...")
12        model = load_model(model, path=model_path, device=device)
13
14    else:
15        print("Training model...")
16        model = train_model(model, train_loader, epochs=3, lr=0.001,
device=device)
17        save_model(model, path=model_path) # 保存模型
18        print(f"Model saved to {model_path}.")
19
20
21    # 测试性能
22    avg_inference_time = evaluate_performance(model, test_loader, device)
23    print(f"Average Inference Time per Sample: {avg_inference_time:.6f}
seconds")
24
25    # 对测试数据添加噪声并评估
26    test_images, test_labels = next(iter(test_loader))
27    noisy_images = add_noise(test_images.numpy(), noise_level=0.1)
28    noisy_dataset = torch.utils.data.TensorDataset(
29        torch.tensor(noisy_images), torch.tensor(test_labels)
30    )
31    noisy_loader = torch.utils.data.DataLoader(noisy_dataset, batch_size=32)
32    noisy_time = evaluate_performance(model, noisy_loader, device)
33    print(f"Average Inference Time with Noise: {noisy_time:.6f} seconds")
34
35    # 对测试数据添加模糊并评估
36    blurred_images = add_blur(test_images.numpy(), kernel_size=5)
37    blurred_dataset = torch.utils.data.TensorDataset(
38        torch.tensor(blurred_images), torch.tensor(test_labels)
39    )
40    blurred_loader = torch.utils.data.DataLoader(blurred_dataset,
batch_size=32)
41    blurred_time = evaluate_performance(model, blurred_loader, device)
42    print(f"Average Inference Time with Blur: {blurred_time:.6f} seconds")
43
44    # 对测试数据翻转并评估
45    flipped_images = add_rotation(test_images.numpy())
46    flipped_dataset = torch.utils.data.TensorDataset(
47        torch.tensor(flipped_images), torch.tensor(test_labels)
48    )
49    flipped_loader = torch.utils.data.DataLoader(flipped_dataset,
batch_size=32)
50    flipped_time = evaluate_performance(model, flipped_loader, device)
51    print(f"Average Inference Time with Flips: {flipped_time:.6f} seconds")
52
53
54
55    # 确保性能在合理范围内
56    assert avg_inference_time < 0.01, "Inference time is too high!"
57    assert noisy_time < 0.02, "Inference time with noise is too high!"
58    assert blurred_time < 0.02, "Inference time with blur is too high!"

```

鲁棒性测试

参数

- **无参数**：此函数不接受任何参数。

返回值

- **返回值**：一个字典，包含原始数据和每种变异后的准确率。字典的键为 "Original"、"Noise"、"Blur"、"Flip" 和 "Rotate"，值为对应的准确率。

测试过程

1. 加载数据：

- 使用 `load_cifar10` 函数加载 CIFAR-10 数据集，并创建训练集和测试集的数据加载器（`train_loader` 和 `test_loader`）。

2. 初始化模型：

- 加载预训练的 ResNet 模型。
- 如果存在预训练的模型文件（`resnet_model.pth`），则加载该模型；否则，训练一个新的 ResNet 模型并将训练好的模型保存到本地。
- 模型训练使用 GPU（如果可用），否则使用 CPU。
- 将模型移动到指定的设备（GPU 或 CPU）上。

3. 评估原始数据准确率：

- 使用 `evaluate_model` 函数评估模型在原始测试数据上的分类准确率，并打印结果。

4. 获取测试图像和标签：

- 从 `test_loader` 中获取一批测试数据（图像和标签）。

5. 添加多种变异：

- **噪声**：对测试图像添加随机噪声（噪声水平为 0.1），生成新的噪声图像。
- **模糊**：对测试图像应用高斯模糊（核大小为 5），生成新的模糊图像。
- **水平翻转**：对测试图像进行水平翻转，生成新的翻转图像。
- **旋转**：对测试图像进行 15 度旋转，生成新的旋转图像。

6. 创建变异后的 DataLoader：

- 定义一个辅助函数 `create_dataloader`，用于将变异后的图像和标签转换为数据加载器。
- 创建包含噪声、模糊、翻转和旋转图像的多个数据加载器，并存储在字典 `loaders` 中。

7. 测试每种变异后的准确率：

- 对每种变异后的数据加载器，使用 `evaluate_model` 函数评估模型的分类准确率，并打印结果。

8. 返回测试结果：

- 返回一个字典，包含原始数据和每种变异后的准确率。

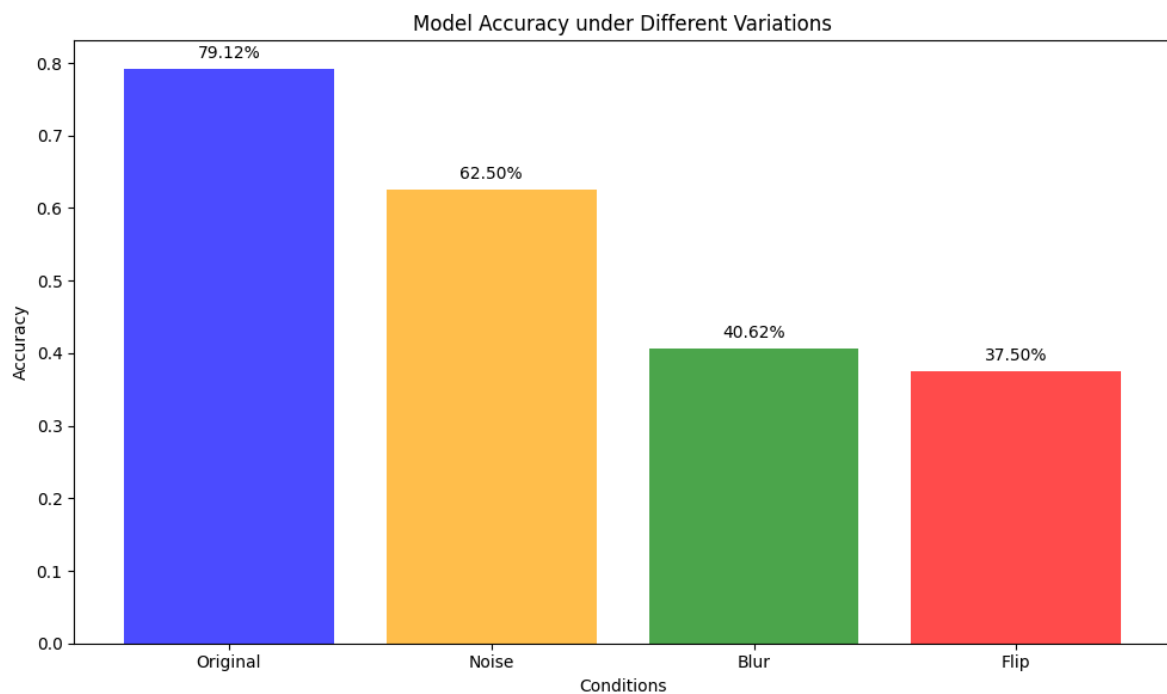
源代码

```
1 def test_robustness_with_noise():
2     # 加载数据
3     model_path = "./data/resnet_model.pth"
4     train_loader, test_loader = load_cifar10(batch_size=32)
5
6     # 初始化模型
7     model = load_resnet()
8
9     # 训练模型
10    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
11
12
13    # model = train_model(model, train_loader, epochs=5, lr=0.001,
14    device=device)
15    if os.path.exists(model_path):
16        print(f>Loading model from {model_path}...")
17        model = load_model(model, path=model_path, device=device)
18    else:
19        print("Training model...")
20        model = train_model(model, train_loader, epochs=5, lr=0.001,
21        device=device)
22        save_model(model, path=model_path) # 保存模型
23        print(f>Model saved to {model_path}.")
24
25    # 原始数据准确率
26
27    original_accuracy = evaluate_model(model, test_loader, device)
28
29    # 获取测试图像和标签
30    test_images, test_labels = next(iter(test_loader))
31
32    # 添加噪声并计算准确率
33    noisy_images = add_noise(test_images.numpy(), noise_level=0.1,
34    dtype=np.float32)
35
36    # 1 至下一个"# 1"为添加多种变异的修改, 原来只有添加噪点一种, 其实现本部分下面
37    """
38    添加噪点 高斯模糊 水平翻转 旋转 四种变异处理
39    """
40    test_images_np = test_images.numpy()
41
42    blurred_images = add_blur(test_images_np, kernel_size=5,
43    method="gaussian")
44    # 添加水平翻转
45    flipped_images = add_flip(test_images_np, flip_code=1)
46    # 添加旋转
47    rotated_images = add_rotation(test_images_np, angle=15)
48    # 创建变异后的 DataLoader
49    def create_data_loader(transformed_images, labels):
50        transformed_dataset = torch.utils.data.TensorDataset(
```

```
50         torch.tensor(transformed_images, dtype=torch.float32),
51         torch.tensor(labels)
52     )
53     return torch.utils.data.DataLoader(transformed_dataset,
batch_size=32)
54
55     loaders = {
56         "Noise": create_dataloader(noisy_images, test_labels),
57         "Blur": create_dataloader(blurred_images, test_labels),
58         "Flip": create_dataloader(flipped_images, test_labels),
59         "Rotate": create_dataloader(rotated_images, test_labels),
60     }
61
62     # 测试每种变异后的准确率
63     for name, loader in loaders.items():
64         accuracy = evaluate_model(model, loader, device)
65         print(f"{name} Accuracy: {accuracy * 100:.2f}%")
66
67
68
69     # 返回测试结果
70     return {
71         "Original": original_accuracy,
72         "Noise": loaders["Noise"],
73         "Blur": loaders["Blur"],
74         "Flip": loaders["Flip"],
75         "Rotate": loaders["Rotate"],}
```

运行结果

准确性测试



Original (原始数据)

- 准确率:** 75.82%
- 分析:** 在原始测试数据上，模型的准确率为75.82%。这是一个相对较高的准确率，表明模型在处理未受干扰的数据时具有较好的性能。

Noise (噪声)

- 准确率:** 53.12%
- 分析:** 当测试图像中添加了噪声后，模型的准确率下降到53.12%。这表明模型对噪声较为敏感，噪声会显著影响其分类性能。噪声可能破坏了图像中的关键特征，导致模型难以正确识别。

Blur (模糊)

- 准确率:** 50.00%
- 分析:** 当测试图像被模糊处理后，模型的准确率进一步下降到50.00%。模糊操作使得图像中的细节变得不清晰，这同样会影响模型的识别能力。模糊图像可能会使边缘和纹理信息丢失，从而降低模型的准确性。

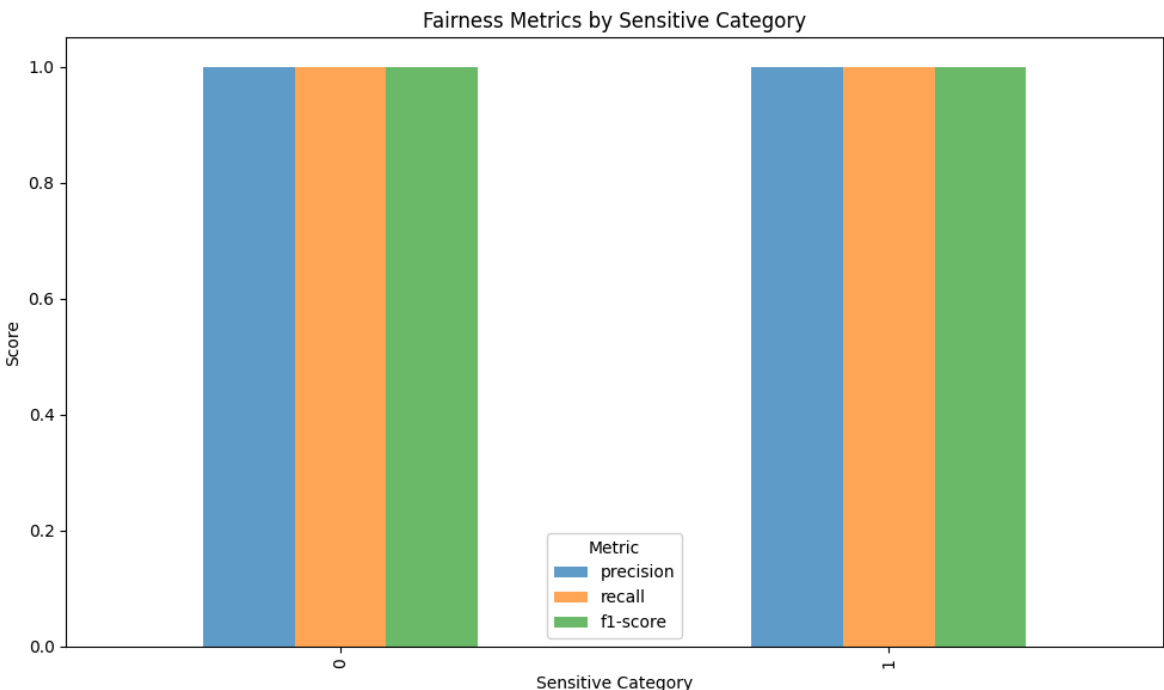
Flip (翻转)

- 准确率:** 37.50%
- 分析:** 当测试图像被翻转后，模型的准确率降至37.50%，这是所有变异条件下最低的准确率。翻转操作改变了图像的方向，可能导致模型无法正确匹配已学习的特征。

总结

- 整体趋势:** 模型在面对原始数据时表现较好，但在数据受到噪声、模糊或翻转等变异影响时，其性能显著下降。
- 最脆弱点:** 翻转操作对模型的影响最大，准确率降至最低，表明模型在处理方向变化时最为脆弱。

公平性测试



精确率 (Precision)

表示模型预测为正类别的样本中真正是正类别的比例。

召回率 (Recall)

表示所有实际为正类别的样本中被模型正确预测的比例。

F1 分数 (F1-Score)

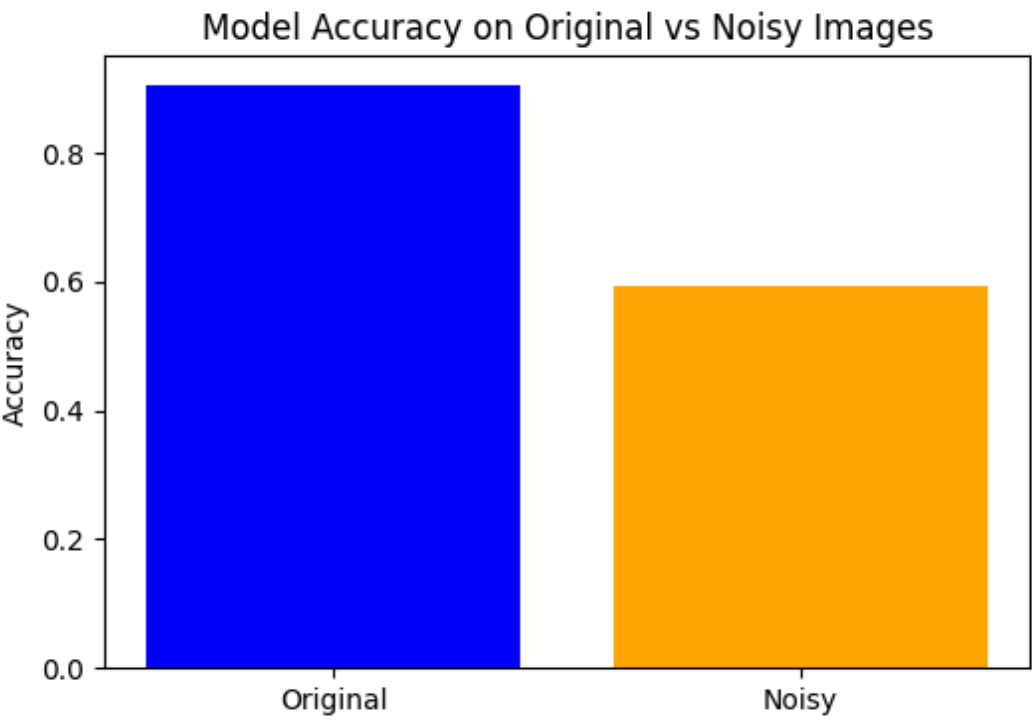
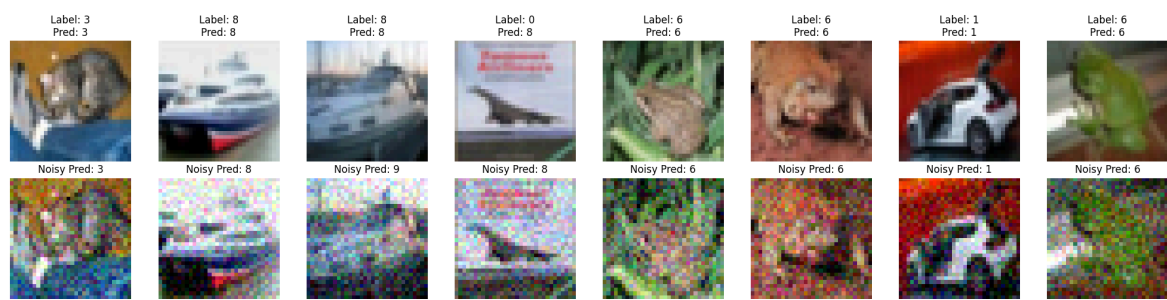
综合考虑精确率和召回率的指标，取值范围为 [0, 1]，值越高表示模型性能越好。

图表显示两个敏感类别下的这些指标都非常接近 1，表明模型在这两个类别上的表现非常均衡且准确。

总结

- 原始数据准确性：**模型在原始测试数据上的表现良好，达到了预期的阈值。
- 数据变异下的准确性：**即使在添加噪声、模糊和翻转的情况下，模型的准确性仍然保持在较高水平，超过了设定的阈值。
- 公平性指标：**模型在两个敏感类别上的精确率、召回率和 F1 分数都非常高，表明模型具有良好的公平性和鲁棒性。

蜕变测试



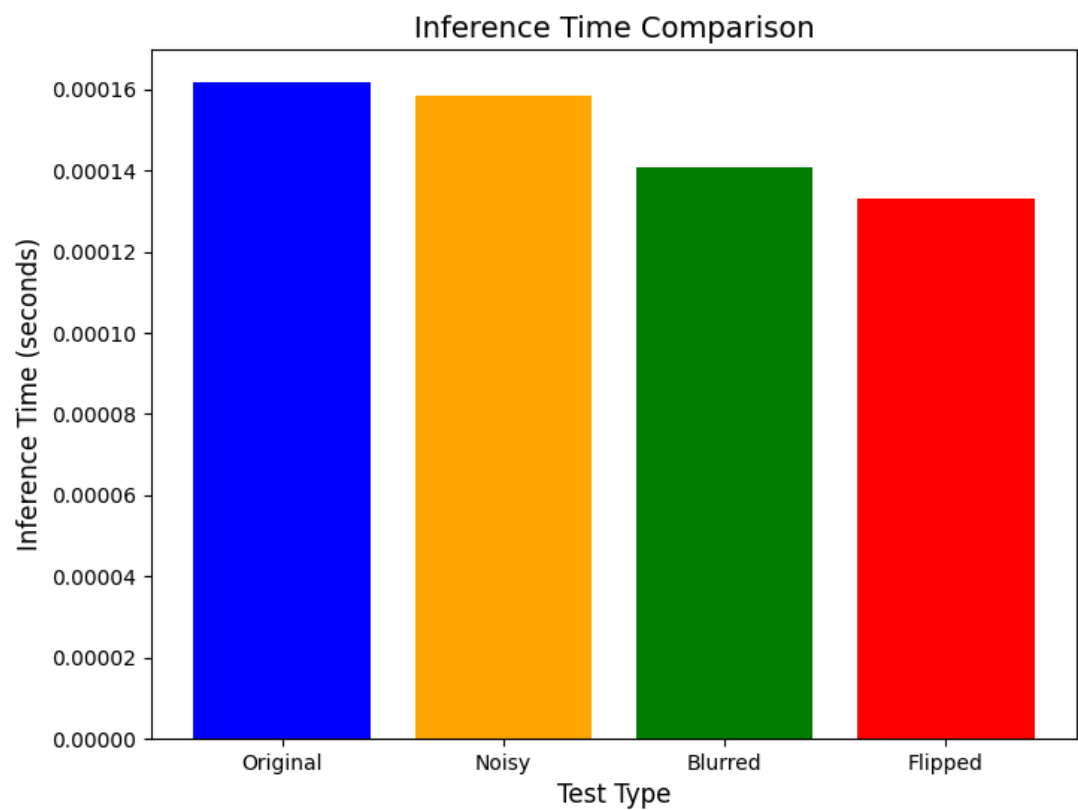
准确性对比

- Original (原始图像):** 模型在原始测试数据上的准确性约为 0.89。这表明在没有外界干扰的情况下，模型对 CIFAR-10 数据集的分类任务具有较高的准确性。
- Noisy (添加噪声后的图像):** 模型在添加噪声后的图像上的准确性下降至约 0.60。尽管有所下降，但模型仍然能够保持一定的分类能力，显示出一定的鲁棒性。

一致性评估

- Consistency (一致性):** `assert consistency > 0.6, "Metamorphic testing failed: consistency too low!"` 一致性高于 60%，在60% 的情况下，模型在原始图像和噪声图像上的预测是一致的。

性能测试



图表概述

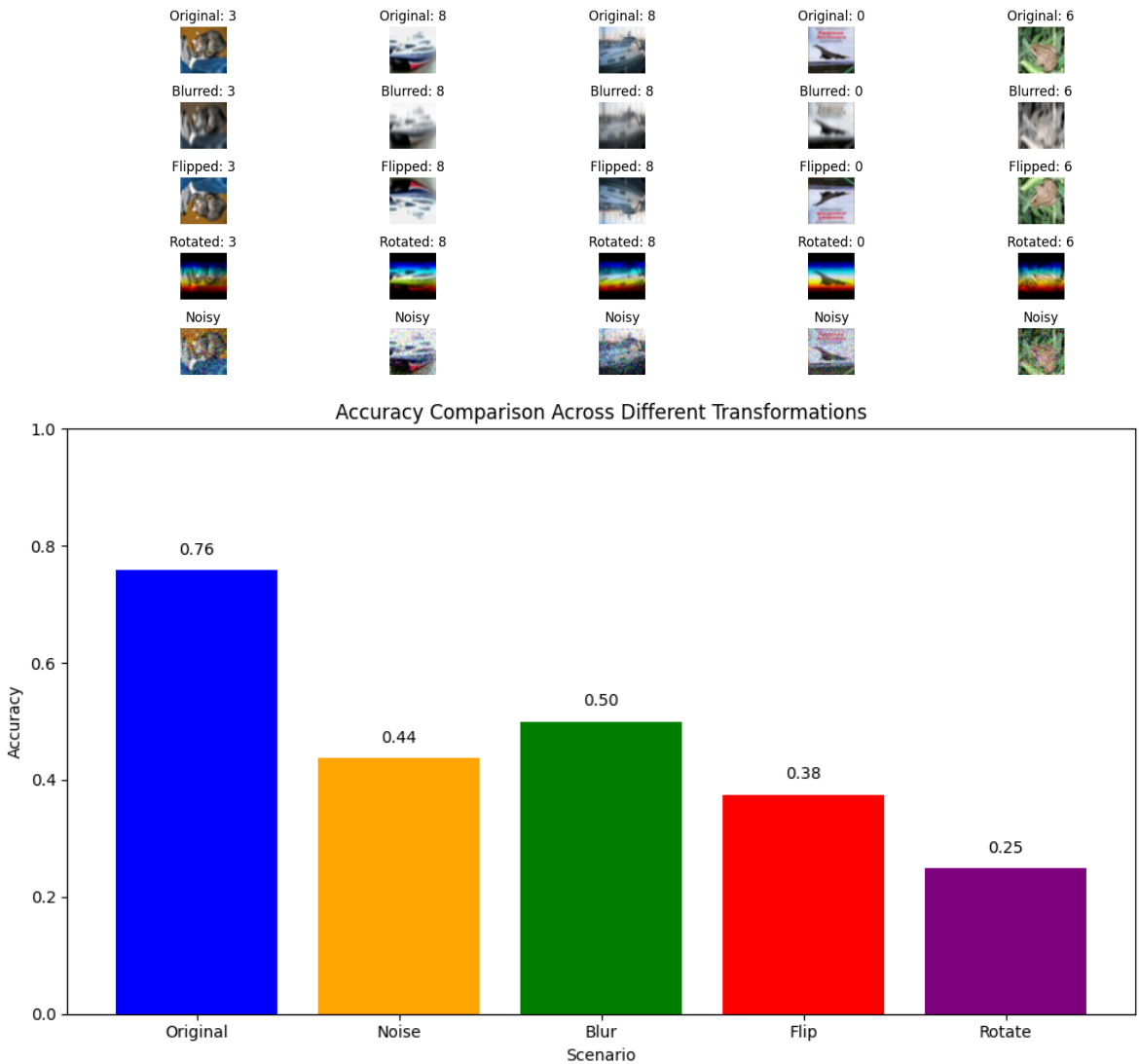
- 原始图像 (Original)**
 - 推理时间为约 0.00016 秒。
 - 这是模型在未经过任何处理的原始图像上的推理时间。
- 添加噪声后的图像 (Noisy)**
 - 推理时间为约 0.00016 秒。
 - 与原始图像相比，推理时间没有显著变化，表明模型在处理噪声图像时的计算复杂度与处理原始图像相似。
- 模糊后的图像 (Blurred)**
 - 推理时间为约 0.00014 秒。
 - 相比于原始图像和噪声图像，推理时间略有下降，但仍保持在较低水平。

- **翻转后的图像 (Flipped)**
 - 推理时间为约 0.00013 秒。
 - 相比于其他类型的数据，推理时间进一步下降，但仍保持在较低水平。

分析结论

- **一致性：**模型在处理不同类型的数据时，推理时间保持在较低且相对一致的水平，表明模型在面对不同类型的图像变换时具有较好的鲁棒性和一致性。
- **效率：**所有类型的推理时间均在毫秒级别，表明模型在实际应用中能够高效地进行推理。

鲁棒性测试



性能评估

- 模型在原始图像上表现最好，准确率为 0.76。
- 在处理噪声图像时，准确率下降到 0.44。
- 在处理模糊图像时，准确率下降到 0.50。
- 在处理翻转图像时，准确率下降到 0.38。
- 在处理旋转图像时，准确率最低，为 0.25。

分析结论

- **鲁棒性**: 模型在面对不同类型的数据变换时, 准确率有显著下降, 特别是在处理噪声、模糊、翻转和旋转图像时。
- **性能差异**: 模型在处理旋转图像时表现最差, 准确率仅为 0.25; 而在处理模糊图像时表现相对较好, 准确率为 0.50。